

杂题选讲

赵晟昊

2026 年 05 月 20 日

题目描述

有 $2m + 1$ 种物品，重量分别为 $-m, -m + 1, \dots, m - 1, m$ 。重量为 i 的物品有 a_i 个。

现在你可以拿走一些重量和恰好为 l 物品，求最多可以拿多少个物品。

$1 \leq m \leq 300, |l| \leq 10^{18}, a_i \leq 10^{12}$ 。

分析

不妨先把所有重量为负数的物品都拿了，然后将其看成个数为 -1 的物品。

先按找性价比排序拿物品，直到再拿一个物品重量和就超过 l 了。考虑这样的方案与最优解的对称差，注意到：最优方案和该方案只会差不超过 $2m$ 个物品。

证明. 考虑每次加入一个物品或者删除一个物品，直到得到最优方案。设当前重量和为 V ，如果每次 $V \geq l$ 就加入一个物品，否则删除一个物品，则任意时刻 V 都在 $[l - m, l + m]$ 内。

注意到我们肯定不会两次经过同一个 V ，因为此时中间一段重量和为 0 ，而我们一开始是按性价比选的，所以这段性价比肯定不超过 0 ，总是不优。 $\square$

那么保留 $[l - m^2, l + m^2]$ 的范围，从贪心方案开始做一个多重背包 DP 即可。

题目描述

有一个 r 行 c 列的 01 矩阵 $s_{x,y}$ ，下标从 1 开始标号。

令 $a_{x,y} = \sum_{j=1}^y s_{x,j}$, $b_{x,y} = \sum_{i=1}^x s_{i,y}$ ，一个格子 (x,y) 的标号定义为 $(a_{x,y}, b_{x,y})$ 。一个 01 矩阵是好的，当且仅当所有 $s_{x,y} = 1$ 的格子的标号互不相同。

现在给你一个矩阵 s ，你可以把其中一些位置从 0 改成 1，问最少改几个位置能让这个矩阵变成好的。

注意不能把元素从 1 变成 0，并且全是 1 的矩阵是好的，所以一定存在解。

$1 \leq r, c \leq 300$ 。

观察

通过手玩 + 猜测，可以得到下面这两个猜想：

猜想 1 不存在一个 0 ，满足正下方和正右方都有 1 。即不存在 $\begin{pmatrix} 0 & 1 \\ 1 & ? \end{pmatrix}$ 作为子矩阵。

猜想 2 不存在 $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ 作为子矩阵。

通过打表可得，这两个猜想都是对的。

相信大家也不想看一大堆证明，所以详细的证明被放在了 [我的博客](#)。在这里我们直接认为它是对的。

推论

如果猜想 1 成立，可得推论：

推论 1.1 设 $x_1 < x_2, y$ 满足 $s_{x_1, y} = s_{x_2, y} = 1$ ，则 $a_{x_1, y} \geq a_{x_2, y}$ 。

证明显然。对于 b 也有类似的推论。

推论 1.2 对于 (x, y) 满足 $s_{x, y} = 1$ ，有 $\forall p \in [1, a_{x, y}], q \in [1, b_{x, y}]$ ，存在标号为 (p, q) 的格子。

证明. 归纳。考虑 (x, y) 上方第一个格子和左方第一个格子，由推论 1.1 可得其分别能覆盖 $[1, a_{x, y}] \times [1, b_{x, y} - 1]$ 和 $[1, a_{x, y} - 1] \times [1, b_{x, y}]$ ，拼上 $(a_{x, y}, b_{x, y})$ 可得能覆盖 $[1, a_{x, y}] \times [1, b_{x, y}]$ 。□

分析

根据 猜想 1 和 猜想 2，我们可以得到这么一个判断合法的方式：

引理 1 一个矩阵是好的，当且仅当**去掉空行空列后**，存在 (x, y) ，满足 $[1, x] \times [1, y]$ 全是 1， $[x + 1, n] \times [y + 1, m]$ 全是 0， $[1, x] \times [y + 1, m]$ 和 $[x + 1, n] \times [1, y]$ 是独立的好矩阵。

分析

根据上面这个判定方式，可以写出一个 $\Theta(n^6)$ 的 DP：记 $dp_{lx,rx,ly,ry}$ 表示把 $[lx, rx] \times [ly, ry]$ 这个子矩形变成好的，最少要几步。

注意到 $[lx, n] \times [ly, m]$ 这个矩形内只有 $[lx, rx] \times [ly, ry]$ 有值，则只要关心 lx, ly 。

接着注意到如果可以选 (x, y) ，且也可以选 $(x, y - 1)$ 或 $(x - 1, y)$ ，则肯定不会选 (x, y) ，其中可以选指的是 $[x + 1, n] \times [y + 1, m]$ 都是 0。这是因为，发现选 $(x, y - 1)$ 或 $(x - 1, y)$ 会确定更少的 1，而选 (x, y) 合法的选 $(x, y - 1)$ 或 $(x - 1, y)$ 仍然可以合法。

这样一个状态只会有 $\Theta(n)$ 种转移，直接做就好了。复杂度 $\Theta(n^3)$ 。

题目描述

Alice 和 Bob 在玩一个游戏，规则如下：

- Alice 手中有一个正整数集 S ，Bob 手中有一个正整数集 T 。
- Alice 和 Bob 依次如下操作（Alice 先手）：选择在自己数集中的两个不同的数字 x, y ，需要满足 $|x - y|$ 不属于对方的数集，然后将 $|x - y|$ 加入对方的数集。最终无法操作的人失败。
- 可以注意到在游戏的进行过程中一个人手中的数集是会不断变化的，他在选择数字 x, y 时既可以选择初始自己拥有的数字，也可以选择后面新增的数字。

现在，给定一个大小为 n 的正整数集 R ，对于所有 S, T 都是 R 的子集的 2^{2n} 种情况，求有多少种情况使得 Alice 必胜。

$1 \leq n \leq 5 \times 10^4$ ， R 中元素在 $[1, 5 \times 10^4]$ 内。

观察

如果 $|S| - 1 > |T|$ ，设 S 中最大值为 mx ，那么将 mx 减去 S 中的每个数就能得到 $|S| - 1$ 个不同数，故 Alice 一定可以操作，此时 Alice 必胜。

同理，如果 $|S| < |T|$ ，那么 Bob 必胜。接下来只需要 $|S| = |T|$ 和 $|S| = |T| + 1$ 的情况，先考虑 $|S| = |T| + 1$ 的情况。

此时 $|S|$ 想要无法操作，必须满足 S 是等差数列，且 T 刚好把 S 卡住，即最终 $S = \{x, x + d, \dots, x + kd\}$, $T = \{d, 2d, \dots, kd\}$ 。

先不管 S 一开始就无法操作的情况，则 x 一定是 d 的倍数，设 $x = ld$ ，则 $S = \{ld, (l + 1)d, \dots, (l + k)d\}$ 。

可以发现 $kd, (k + 1)d, \dots, (l + k)d$ 都是无法由 T 操作得到的，故这些一开始就都在 S 里。由 $l \geq 1$ 可得 $(l + k)d$ 和 $(l + k - 1)d$ 总是在 S 里。

更进一步，发现只要 $(l + k)d$ 和 $(l + k - 1)d$ 在 S 里，Alice 就总是会输。这是因为 T 总是无法操作出 $(l + k)d$ 和 $(l + k - 1)d$ ，等效于 $|T| > |S| + 1$ ，即 T 总是可以操作。

分析

于是, $|S| = |T| + 1$ 时 Alice 赢当且仅当满足下面条件中的一个:

- 存在正整数 x, d, k 使得 $S = \{x, x + d, \dots, x + dk\}, T = \{d, 2d, \dots, dk\}$ 。
- 存在正整数 d, k 使得 $S \subset \{d, 2d, \dots, (k+1)d\}, T \subset \{d, \dots, kd\}$ 且 $kd, (k+1)d \in S$ 。

同理, $|S| = |T|$ 时也是类似的, 由于 S 必须要操作一次, 只有第二种情况。

接下来考虑怎么计数, 显然也只要考虑 $|S| = |T|$ 和 $|S| = |T| + 1$ 的部分。

对于第一种情况, 可以通过枚举 d, k 后用 bitset 维护, 复杂度 $O\left(\frac{n^2 \log n}{w}\right)$ 。

对于第二种情况, 枚举 d, k , 则只要求 $\sum_{i=0}^{k-1} \binom{k-1}{i} \binom{k}{i+1}$ 和 $\sum_{i=0}^{k-1} \binom{k-1}{i} \binom{k}{i+2}$, 使用范德蒙德卷积可得其分别为 $\binom{2k-1}{k-1}$ 和 $\binom{2k-1}{k-2}$ 。

题目描述

给定 n, m 和长度为 n 的序列 $a_1, a_2, \dots, a_n$ ，以及 m 个区间 $[l_1, r_1], [l_2, r_2], \dots, [l_m, r_m]$ ，定义长度为 m 的序列 b 满足 $b_i = \max_{l_i \leq j \leq r_i} a_j$ ，你可以随意重排 a ，求字典序最大的 b 。

$1 \leq n, m \leq 10^5$ 。

分析

从大到小枚举 x ，把取到 x 的区间从前往后写成一个二进制数，每次都要求这个二进制数尽量大。

先求出一个极长前缀取到 x （这个可以通过倍增加二分实现），则后面每个点取值都在一个不交的区间 $[L_i, R_i]$ 内，且每个 $[L_i, R_i]$ 中的每个点都能取到。

记 f_i 表示若 i 位置放了一个 x ，则下一个 x 最远能放到 f_i ，同理 g_i 表示往前最远能放到 g_i ，则 $R_i = f_{R_{i-1}}$ ， $L_i = g_{L_{i+1}}$ 。 f, g 可以用线段树维护，则 L_i, R_i 可以直接计算出来。

分析

每次找到下一个和任意一个 $[L, R]$ 有交的区间 $[l, r]$ ，则这个 $[l, r]$ 一定能取到当前的 x ，考虑其对 f, g, L, R 的影响：

- 如果和 ≥ 3 个 $[L, R]$ 有交，则总是能取到 x ，且对 f, g 没有影响。
- 如果和 2 个区间有交，则不会改变 L, R ，更新一下 f, g 即可。
- 如果和 1 个区间有交，则会改变 L, R 和 f, g 。

考虑怎么找到每个有交的 $[l, r]$ ，可以用标记永久化线段树维护出所有和每个 $[L, R]$ 有交的 $[l, r]$ ，然后用一个堆维护标号最前的。由于如果一个区间和至少三个 $[L, R]$ 有交就必定能取到 x ，所以每个 $[l, r]$ 只会被放进堆里两次。

注意到：改变 L, R 的次数总和为 $O(n)$ ！这是因为 L, R 总是一个 $[l, r]$ 的端点，所以每次改变都会永久删掉一个端点，最多删掉 $O(n)$ 次。于是， L_i, R_i 可以暴力处理。

题目描述

给定一棵 n 个点的有根树，满足 $1, 2, \dots, n$ 为该树的一个 DFS 序。

对于 $c = 1, 2, \dots, n$ ，求长度为 c 的序列 $a_1, \dots, a_c$ 个数，满足：

- 不存在 $1 \leq i < j \leq c$ ，使得 a_j 在 a_i 的子树内。
- 对于 $1 \leq i \leq c$ ， $a_i \leq n - i + 1$ 。

答案对 998244353 取模。

$n \leq 80$ 。

分析

考虑给每个数分配一个 a 中的位置。

对于一个子树 $[l, r]$ ，注意到只有把里面的点放在 $a_{n-r+1}, \dots, a_{n-l+1}$ 才可能违反第二条限制，放在 $a_1, \dots, a_{n-r}$ 则总是不会违反。

对于第一条限制，显然只需要考虑每个子树内部的相对顺序。

记 $f_{u,c,i,j}$ 表示满足以下的方案数：

- 只考虑 u 子树内的数，且所有数只能填在 a 的 $[n-r+1, n-r+c]$ 内。
- 有 i 个数会被填在 a 的 $[1, n-r]$ 内，这些数的相对顺序已经确定了，但具体填在哪里还没确定。
- 剩下有 j 个数被填在了 $[n-r+1, n-r+c]$ 内，每个数的具体位置都已经确定，剩下了 $c-j$ 个空位。

考虑每次合并两个子树 $[l, m], [m+1, r]$ ，只需要枚举 $[l, m]$ 中填到 $[1, n-m]$ 的点有几个位于 $[n-r+1, n-m]$ ，剩下两边在 $[1, n-r]$ 的点显然可以任意排列。

分析

先把所有子树合并起来，最后加入 u 。有若干种可能：

- 不选 u 。
- 把 u 放在 $[n - r + 1, n - l + 1]$ 之中。
- 把 u 放在 $[1, n - r]$ 的最后一个。此时要求 $[n - r, n - l]$ 没有数。

直接做复杂度是是 $O(n^6)$ 的，合并时需要枚举两边 i, j ，以及最后的 c ，再枚举之前的放在当前区间的有几个。

发现贡献可以写成二维卷积的形式，可以插值优化到 $O(n^5)$ 。

更好的做法

注意到只有 a 的后缀最大值需要考虑第二条限制。

考虑从 n 到 1 把每个数填到 a 中，则成为后缀最大值当且仅当填到了所有 a 当前已填数的后面。

$f_{i,j,k}$ 表示，已经考虑完 $[i+1, n]$ 内的点，当前 a 中最后一个后缀最大值是 j ， $[1, j]$ 填了 k 个数的方案数。转移：

- 没填 i 。从 $f_{i+1,j,k}$ 转移。
- 填了 i ，是后缀最大值。从 $f_{i+1,j',k+1}$ 转移，其中 $j < j' \leq n - i + 1$ 。
- 填了 i ，不是后缀最大值。从 $f_{i+sz[i],j,k}$ 转移，枚举子树选了几个。预处理 $g_{i,j}$ 表示 i 子树内选 j 个，要求儿子在父亲之前的排列方案数。

复杂度 $O(n^4)$ 。

题目描述

有一个小写字母组成的字符串 S ，建立一张有向图 G ，使得：

- G 的每个顶点都是 S 的子串。
- 对于两个 S 的子串 s_1, s_2 ，若存在字符 c 使得 $s_2 = s_1 + c$ 或者 $s_2 = c + s_1$ ，就让 s_1 向 s_2 连一条边。
- 对于所有重边只保留一条。

现在给你 G 的结构（打乱点的标号），求字典序最小的 S 。保证有解。

设 G 的点数和边数分别为 n, m ，则 $1 \leq n \leq 10^6, 0 \leq m \leq 2 \times 10^6$ 。

分析

注意到：

1. 如果按长度分层，每条边只在相邻两层之间。可以求出每个点所在层。
2. 没有出度的点对应 $[1, n]$ 。每个点有不超过两个入度。
3. 把原来的字符串 S 翻转后，显然 G 的结构不变。

尝试对于每个 $[l, r]$ ，判断它的两个入边中哪个是 $[l, r - 1]$ ，即判断它们的顺序。显然 $[1, n]$ 的入度无法区分，但由于翻转 S 后图不变，所以不管认为哪个是 $[1, n - 1]$ 都是有解的，随意确定一个即可。

对于一个节点 $[l, r]$ 的两个入边 $[l, r - 1], [l + 1, r]$ ，如果这两个点的入边交集为 1，则根据 $[l, r - 1], [l + 1, r]$ 的顺序即可推出其入边的顺序。由于 $[1, n]$ 的入边顺序已经确定，所以可以按长度从大到小考虑每个点。

同一层内，先将这样交集为 1 的情况处理，对于剩下的交集为 2 的情况，可以发现除非这一层只有这一个点， $[l, r - 1]$ 和 $[l + 1, r]$ 中至少会有一个点已经确定入边顺序，故也可以简单判断。

分析

现在已经能得到每个节点的入边顺序，考虑由此得到一个等价关系。

仍然是按长度从大到小考虑，用并查集维护，若 x, y 在同一个连通块则 $[x, x + len - 1] = [y, y + len - 1]$ ，这样每次 len 减少时不会破坏原来的等价关系。

对于每个节点 u ，维护其对应的等价类中最小的那个，记为 L_u ，每次把 L_u 和 $L_u + 1$ 按顺序合并到两个入边的 L 即可。

题目描述

给定 n, m ，有一棵 $n \times m + 1$ 个点的树，顶点编号为 $0, 1, \dots, nm$ ，第 i 条边 ($1 \leq i \leq nm$) 连接 i 和 $\max(i - n, 0)$ 。

初始，第 i 个点的颜色为 $i \bmod n$ 。你可以执行任意次操作，每次操作形如：

- 选定通过一条边相连的两个点 u, v ，将 u 的颜色改为 v 的颜色。

求一共能得到多少种不同的树。答案对 998244353 取模。

$n, m \leq 2 \times 10^5$ 。

为了让大家在思考时间有一点事干，我们给出一些提示（即第一步转化）：

提示

考虑从最终状态出发，判断其能否到达初始状态。每次可以执行的操作为：选择一个有同色邻居的点 u ，将 u 改成任意的颜色。

将两端颜色相同的边称为**好边**，按**好边的数量**分类讨论。

观察

显然我们可以任意移动好边。

如果现在除了根相连的边，其余边都是好边，且每条链的颜色互不相同，那么只需要交换两条链的颜色。

更进一步，如果除了根相连的边都是好边，但是每条链的颜色可能相同，先尝试让根连出去至少一条好边（可以通过将一条链开头的好边移动到根来实现），然后设 x 为至少出现在两条链的颜色，将根改为 x ，并将其中一条链设为没出现过的颜色，此时根仍然连着好边，可以继续操作，以此类推。

于是，只要好边够多就赢了，问题在于怎么增加好边的数量。

观察

称深度为 i 点为第 i 层。

1. 如果有两个第一层的颜色相等，且有一条和根相连的不是这个颜色的好边，那么直接把根改成这个颜色就能得到两条好边。这样的操作并不是没有代价的，代价就是第一层的颜色数会加一。
2. 如果有一个第一层的点和一个不同链的第二层的点颜色相同，且这两条链外有两个好边，可以先把一个好边移动到第二层的点那儿，然后用剩下那个好边执行第一步的过程，这样就能得到一个额外的好边。
3. 如果有至少三个好边，可以不停制造出第二步的情况，从而获得大量好边。多余的好边可以放到链的深处，所以可以一直这么操作直到所有非根边都是好边。

由上，可以得到，如果由三个好边就合法。显然一到两个好边的情况能操作得到三个好边也能合法，接下来考虑什么样的情况能得到三个好边。

观察

首先没有好边根本就操作不了，所以不用管。

其余情况，先把好边移到第一层。考虑第一层的颜色数：

1. 如果第一层有 n 种颜色，说明根本无法得到新的好边，不合法。
2. 如果第一层有 $n - 1$ 种颜色，那么可以得到至少两条好边，只需要考虑这种情况。
3. 如果第一层不到 $n - 2$ 种颜色，那么可以得到至少三条好边，直接合法。

对于第一层有 $n - 1$ 种颜色的情况，发现大部分时候都能找到一个第一层的点和一个不同链的第二层的点颜色相同，然后就能得到一个新的好边，除非第二层全是剩下的第 n 种颜色。

分析

总结：绝大多数情况都合法，除了以下情况：

1. 没有好边。
2. 一条好边，将好边移到第一层后，第一层有 n 种颜色。
3. 一/二条好边，将好边移到第一层后，第一层有 $n - 1$ 种颜色，且第二层全是剩下那种颜色。

用全部方案数减去上述不合法方案数即可。

需要注意的是，根据以上分析， $n \leq 2$ 或者 $m = 1$ 需要一些简单的特判。